APUNTES UNIVERSITARIOS

Foundations of Machine Learning and Deep Learning

Apuntes del Tema 6: Regularización en Redes Profundas

18 de octubre de 2026

Índice general

1 Apuntes del Tema 6: Regularización en Redes Profundas **2**

1.1 1. El Problema del Sobreajuste (*Overfitting*) en Redes Profundas 2

1.1.1 1.1 Naturaleza del Problema 2

1.2 2. Decaimiento de Pesos (*Weight Decay* / Regularización L_2 vs. L_1) 2

1.2.1 2.1 Decaimiento de Pesos (L_2 Regularization) 2

1.2.2 2.2 Regularización L_1 (Lasso) 3

1.3 3. Dropout (Inyección de Ruido en Activaciones) 3

1.3.1 3.1 Mecanismo de Funcionamiento 3

1.3.2 3.2 ¿Por qué funciona Dropout? 3

1.4 4. Parada Temprana (*Early Stopping*) 4

1.4.1 4.1 Criterio de Funcionamiento 4

1.5 5. Práctica y Aplicación en Código (PyTorch) 4

1 Apuntes del Tema 6: Regularización en Redes Profundas

1.1 1. El Problema del Sobreajuste (*Overfitting*) en Redes Profundas

1.1.1 1.1 Naturaleza del Problema

- **Alta Capacidad Expresiva:** Las redes neuronales profundas poseen millones de parámetros libres. Esta gran capacidad les permite memorizar no solo los patrones subyacentes de los datos, sino también el ruido de entrenamiento.
- **Brecha de Generalización (*Generalization Gap*):** Se define como la diferencia entre la pérdida/error en el conjunto de validación (o prueba) y el conjunto de entrenamiento.
- *Baja varianza / Buen ajuste:* La pérdida de validación disminuye a la par con la de entrenamiento.
- *Alta varianza / Overfitting:* A partir de cierto punto, la pérdida de entrenamiento sigue bajando mientras la pérdida de validación empieza a crecer, ensanchando la brecha de generalización.
- **Objetivo de la Regularización:** Modificar o restringir el proceso de aprendizaje para reducir el error de prueba/generalización sin incrementar excesivamente el error de entrenamiento.

1.2 2. Decaimiento de Pesos (*Weight Decay* / Regularización L_2 vs. L_1)

La forma clásica de regularización consiste en añadir un término de penalización $\Omega(W)$ a la función de pérdida original $\mathcal{L}(W)$:

$$\mathcal{J}(W) = \mathcal{L}(W) + \lambda\Omega(W)$$

donde $\lambda > 0$ es el hiperparámetro que controla la intensidad de la regularización.

1.2.1 2.1 Decaimiento de Pesos (L_2 Regularization)

- **Penalización:** Se utiliza la norma L_2 al cuadrado de las matrices de pesos:

$$\Omega(W) = \frac{1}{2}\|W\|_2^2 = \frac{1}{2}\sum_i \sum_j w_{ij}^2$$

- **Efecto en el Gradiente y Actualización (SGD):**

$$\frac{\partial \mathcal{J}}{\partial W} = \frac{\partial \mathcal{L}}{\partial W} + \lambda W$$

$$W \leftarrow W - \eta\frac{\partial \mathcal{J}}{\partial W} = W(1-\eta\lambda) - \eta\frac{\partial \mathcal{L}}{\partial W}$$

- **Interpretación:** Antes de aplicar el paso de gradiente habitual, los pesos se reducen por un factor multiplicativo $(1-\eta\lambda) < 1$. Esto fuerza a la red a mantener pesos pequeños y distribuidos de forma suave, previniendo comportamientos extremos frente a pequeñas variaciones en la entrada.

1.2.2 2.2 Regularización L_1 (Lasso)

- **Penalización:** Suma de los valores absolutos de los pesos:

$$\Omega(W) = \|W\|_1 = \sum_i \sum_j |w_{ij}|$$

- **Efecto:** Mantiene una magnitud de penalización constante $\lambda \cdot \text{signo}(W)$ independientemente del tamaño del peso, lo que empuja muchos pesos **exactamente a cero**, generando matrices de pesos esparsas (*sparse models*) y actuando como selección implícita de características.

1.3 3. Dropout (Inyección de Ruido en Activaciones)

Proposed por Srivastava et al. (2014), es una de las técnicas de regularización más efectivas en redes profundas.

1.3.1 3.1 Mecanismo de Funcionamiento

Durante el entrenamiento, en cada iteración o minilote, cada neurona oculta se "apaga" (se fuerza a cero) con una probabilidad $p \in (0, 1)$ elegida de forma independiente.

- **Formulación Matemática (Inverted Dropout):**

Dada una activación intermedia h, se transforma en h' mediante:

$$h' = \begin{cases} 0 & \text{con probabilidad } p \\ \frac{h}{1-p} & \text{con probabilidad } 1-p \end{cases}$$

- **Reescalado ($\frac{1}{1-p}$):** Escalar por $\frac{1}{1-p}$ durante el entrenamiento garantiza que el valor esperado de la activación sea constante ($E[h'] = h$), evitando modificar la magnitud de las entradas en la fase de test.

```
Antes de Dropout                Después de Dropout (con p = 0.4)

  [x1]  [x2]                      [x1]  [x2]


  [h1]  [h2]                      [ 0 ]  [h2]   <- Neurona h1 desactivada


  [o1]  [o2]                      [o1]  [o2]
```

1.3.2 3.2 ¿Por qué funciona Dropout?

1. **Prevención de la Co-adaptación:** Fuerza a cada neurona a aprender características útiles por sí misma, en lugar de depender de la presencia corregidora de otras neuronas específicas.

1. **Ensamble Implícito (*Implicit Ensembling*):** Entrenar con Dropout equivale a entrenar en cada paso un subconjunto distinto de la red (*thinned network*) que comparte pesos. La evaluación final sin Dropout equivale geométricamente a promediar las predicciones de un número masivo de subredes.

1. **Inferencia / Test:** Durante la fase de evaluación, el Dropout debe **desactivarse** llamando a `model.eval()` en PyTorch.

1.4 4. Parada Temprana (*Early Stopping*)

Es una técnica de regularización no paramétrica que aprovecha la dinámica temporal del entrenamiento.

1.4.1 4.1 Criterio de Funcionamiento

- **Fundamento:** A medida que aumentan las épocas de entrenamiento, el error de entrenamiento disminuye de manera monótona, pero el error de validación alcanza un mínimo y luego empieza a aumentar.

- **Mecanismo:**

1. Se evalúa el error sobre el conjunto de validación al final de cada época.

1. Se guarda una copia de los mejores pesos del modelo cuando la pérdida de validación alcanza un nuevo mínimo.

2. **Criterio de Paciencia (*Patience*):** Si la pérdida de validación no mejora durante un número P determinado de épocas consecutivas, el entrenamiento se interrumpe y se restauran los pesos óptimos guardados.

```
Pérdida

          \         /  <- Pérdida de Validación (Empieza a subir)

           \_______/

              Parada Temprana (Guardar mejor modelo)

             \

              \_______ <- Pérdida de Entrenamiento (Sigue bajando)

                  Épocas
```

1.5 5. Práctica y Aplicación en Código (PyTorch)

A continuación se presenta la implementación modular que integra **Weight Decay** (L_2), **Dropout** y **Early Stopping** utilizando PyTorch:

```
import torch

import torch.nn as nn
```

```
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset


# ------------------------------------------------------------------------
# 1. Definición del Modelo con Dropout
# ------------------------------------------------------------------------
class RegularizedMLP(nn.Module):
    def __init__(self, input_dim=20, hidden_dim=64, num_classes=2, dropout_p=0.5):
        super(RegularizedMLP, self).__init__()

        self.network = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(p=dropout_p),  # Inyección de ruido en activaciones

            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(p=dropout_p),

            nn.Linear(hidden_dim, num_classes)  # Logits de salida
        )

    def forward(self, x):
        return self.network(x)


# ------------------------------------------------------------------------
# 2. Clase para Gestionar Early Stopping
# ------------------------------------------------------------------------
class EarlyStopping:
```

```
63
64      def __init__(self, patience=5, delta=0.001):
65
66          self.patience = patience
67
68          self.delta = delta
69
70          self.best_loss = float('inf')
71
72          self.counter = 0
73
74          self.early_stop = False
75
76          self.best_state_dict = None
77
78
79
80      def __call__(self, val_loss, model):
81
82          if val_loss < self.best_loss - self.delta:
83
84              self.best_loss = val_loss
85
86              self.counter = 0
87
88              self.best_state_dict = model.state_dict().copy() # Guardar mejores pesos
89
90          else:
91
92              self.counter += 1
93
94              if self.counter >= self.patience:
95
96                  self.early_stop = True
97
98
99
100 # ------------------------------------------------------------------------
101
102 # 3. Bucle de Entrenamiento y Validación
103
104 # ------------------------------------------------------------------------
105
106 def train_with_regularization():
107
108     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
109
110
111
112     # Datos sintéticos
113
114     X_train = torch.randn(800, 20)
115
116     y_train = torch.randint(0, 2, (800,))
117
118     X_val = torch.randn(200, 20)
119
120     y_val = torch.randint(0, 2, (200,))
```

```
121
122
123
124     train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=32, shuffle
        =True)
125
126     val_loader = DataLoader(TensorDataset(X_val, y_val), batch_size=32, shuffle=False
        )
127
128
129
130     model = RegularizedMLP(input_dim=20, dropout_p=0.3).to(device)
131
132     criterion = nn.CrossEntropyLoss()
133
134
135
136     # Regularización L2 (Weight Decay) aplicada directamente en el optimizador
137
138     optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
139
140     early_stopper = EarlyStopping(patience=10)
141
142
143
144     epochs = 100
145
146     for epoch in range(1, epochs + 1):
147
148         # FASE DE ENTRENAMIENTO (Dropout Activo)
149
150         model.train()
151
152         train_loss = 0.0
153
154         for images, labels in train_loader:
155
156             images, labels = images.to(device), labels.to(device)
157
158             optimizer.zero_grad()
159
160             outputs = model(images)
161
162             loss = criterion(outputs, labels)
163
164             loss.backward()
165
166             optimizer.step()
167
168             train_loss += loss.item() * images.size(0)
169
170
171
172         train_loss /= len(train_loader.dataset)
173
174
175
176         # FASE DE VALIDACIÓN (Dropout Desactivado)
```

```
177
178         model.eval()  # Importante: desactiva Dropout
179
180         val_loss = 0.0
181
182         with torch.no_grad():
183
184             for images, labels in val_loader:
185
186                 images, labels = images.to(device), labels.to(device)
187
188                 outputs = model(images)
189
190                 loss = criterion(outputs, labels)
191
192                 val_loss += loss.item() * images.size(0)
193
194
195
196         val_loss /= len(val_loader.dataset)
197
198
199
200         print(f"Época [{epoch:3d}/{epochs}] | Train Loss: {train_loss:.4f} | Val Loss:
    {val_loss:.4f}")
201
202
203
204         # Comprobar criterio de Parada Temprana
205
206         early_stopper(val_loss, model)
207
208         if early_stopper.early_stop:
209
210             print(f"\n[Early Stopping] Entrenamiento detenido en la época {epoch}.")
211
212             # Restaurar mejores pesos
213
214             model.load_state_dict(early_stopper.best_state_dict)
215
216             break
217
218
219
220 if __name__ == '__main__':
221
222     train_with_regularization()
```